

CONDITIONAL STATEMENTS

What are Control Statements?

Control statements are like the traffic signals of your code; they direct the flow of execution, deciding which parts of your program run and when. They are essential for making decisions, repeating tasks, and creating complex algorithms.

Python has three main types of control statements:

1. **Conditional Statements** (if, elif, else): For making decisions.
2. **Looping Statements** (for, while): For repeating tasks.
3. **Transfer Statements** (break, continue, pass): For altering the flow within loops.

This summary focuses on **Conditional Statements**.

The if Statement

The if statement is the most basic conditional statement. It executes a block of code **only if** a specified condition evaluates to `True`. The code block must be **indented**. In Python's boolean context, any **non-zero number** or non-empty object is considered `True`, while `0` is considered `False`.

Python

```
# A simple if statement
```

```
x = 10
```

```
if x > 0:
```

```
    print("x is a positive number") # This will be printed
```

```
# An if statement with a False condition
```

```
y = -5
```

```
if y > 0:
```

```
    print("y is a positive number") # This will NOT be printed
```

The if-else Statement

The if-else statement provides an alternative path. The if block runs if the condition is `True`, and the else block runs if the condition is `False`.

Python

```
# Check if a number is even or odd
```

```
number = 7
```

```
if number % 2 == 0:
```

```
    print("The number is Even")
```

```
else:
```

```
    print("The number is Odd") # This will be printed
```

The if-elif-else Statement

This structure allows you to check multiple conditions in sequence. Python will execute the code block for the **first condition** that is `True` and skip the rest. The final else block is optional and runs if none of the preceding conditions are met.

Python

```
# Check if a number is positive, negative, or zero
```

```
num = 0
```

```
if num > 0:
    print("The number is Positive")
elif num < 0:
    print("The number is Negative")
else:
    print("The number is Zero") # This will be printed
```

Nested Conditional Statements

You can place one conditional statement inside another to handle more complex, layered logic. The inner if statement is only evaluated if the outer if statement's condition is True.

Python

Check if a number is positive AND even

num = 10

```
if num > 0:
    print("The number is positive.")
    if num % 2 == 0:
        print("And it is also an even number.")
```

Shorthand Notations (Ternary Operators)

Python offers a more concise way to write simple conditional statements on a single line.

- **Shorthand if:**
Python

```
if 5 > 2: print("Five is greater than two!")
```
- Shorthand if-else (Ternary Operator):

This is a clean way to assign a value to a variable based on a condition. The syntax is `value_if_true if condition else value_if_false`.

Python

age = 20

status = "Adult" if age >= 18 else "Minor"

print(status) # Output: Adult

- Shorthand if-elif-else:

This can be achieved by chaining ternary operators.

Python

score = 75

grade = 'A' if score >= 90 else 'B' if score >= 80 else 'C'

print(grade) # Output: C